

Mock Design: A Food Delivery System

Phase 4 · Interview-Ready · Estimated time: 2 hours

1. Concept From First Principles: Applying the Framework

Food delivery (Swiggy/Zomato/Talabat/HungerStation-style) is deliberately positioned late in this Phase because it's a genuine **combination problem**: it needs Chapter 11's geospatial matching (for delivery partner assignment), Chapter 10's inventory-and-order correctness thinking (for restaurant order management), and an entirely new concept this program hasn't covered — coordinating a **three-party, multi-stage transaction** (customer, restaurant, delivery partner) where each party can fail or delay independently.

Step 1 — Clarify requirements

- Functional: browse restaurants/menus by location; place an order; restaurant accepts/prepares; a delivery partner is assigned, picks up, and delivers; live order-status tracking throughout.
- Non-functional: accurate ETA prediction matters a great deal to user experience; the system must handle the multi-party order lifecycle correctly even when any one party (restaurant, delivery partner) is slow or fails to respond.

Step 2 — Estimate scale

Assume 10 million orders/day, heavily concentrated in lunch and dinner peak windows (a well-documented real characteristic of this domain) — meaning average throughput numbers understate the real design target; peak-window throughput might be 5–8x the 24-hour average. This peaked, bursty load pattern, combined with the geospatial write load from Chapter 11 for delivery partner locations, is the key scale framing for this chapter.

Step 3 — API

POST /orders {customerId, restaurantId, items} -> {orderId, status}, with status progressing through a defined lifecycle: PLACED -> ACCEPTED -> PREPARING -> READY_FOR_PICKUP -> PICKED_UP -> DELIVERED (or CANCELLED at various points). Modeling this explicitly as a **state machine** is itself a strong design signal — it makes clear exactly which transitions are valid and prevents the order from ending up in an inconsistent or undefined state.

Step 4 — High-level design

An Order Service owns the order state machine and coordinates three downstream concerns: a Restaurant Service (order goes into the restaurant's queue, they accept/reject and update prep status), the Matching/Dispatch Service from Chapter 11 (finds and assigns a nearby available delivery partner once the order is near ready), and a Notification Service (Chapter 15) that keeps customer, restaurant, and delivery partner all informed of state changes in real time. Each of these interactions is asynchronous and event-driven (Phase 3, Ch 8) — the Order Service publishes state-change events, and each downstream service reacts independently, rather than the Order Service synchronously calling and waiting on all three.

Step 5 — Deep dive: the multi-party order state machine and ETA prediction

Coordinating three independent parties is the hardest, most differentiating part of this design. Each party can fail or stall independently: a restaurant might not respond to an order within a reasonable window, or no delivery partner might be available nearby. A strong answer names an explicit **timeout-and-fallback** policy at each state transition: if a restaurant doesn't accept within, say, 90 seconds, auto-cancel and refund, or reroute to a similar restaurant; if no delivery partner is found within a window, widen the search radius progressively (directly reusing Chapter 11's geospatial index, just querying a larger area) rather than leaving the order stuck. This is conceptually a **Saga** (Phase 3, Ch 4; Chapter 10) spanning three independent services instead of two, with compensating actions (refund, re-notify, reassign) defined at every step where a party might fail to respond.

ETA prediction genuinely matters to the product experience and is a real, separate subsystem in production (Swiggy's and Zomato's engineering blogs both discuss dedicated ETA-prediction services). For interview purposes, naming the inputs a real system would feed into an ETA model — current restaurant prep-time history, real-time delivery partner distance and traffic conditions, and historical delivery times for the same route/time-of-day — and stating that this would be a separate service consuming live location and historical data (rather than attempting to build the prediction model itself live) is the appropriately-scoped answer.

Step 6 — Trade-offs and wrap-up

Volunteer: peak lunch/dinner load means the system needs to auto-scale aggressively around predictable daily windows rather than provisioning for average load year-round — a cost/complexity trade-off worth naming, and a good opportunity to reference Phase 3 Chapter 13's metrics-driven alerting as the mechanism that would catch a restaurant-acceptance or delivery-matching slowdown during exactly these peak windows before it becomes a customer-visible problem.

2. Real-World Example / Case Study

Swiggy's and Zomato's own publicly discussed platform architecture, and Talabat's and HungerStation's equivalent operations across Gulf markets, all describe this exact three-party coordination challenge — a single order genuinely depends on the independent, uncoordinated actions of a restaurant and a delivery partner, neither of which the platform directly controls, making explicit timeout/fallback handling a hard production requirement, not an interview nicety.

3. Diagram

```

Order state machine:
PLACED -> ACCEPTED -> PREPARING -> READY_FOR_PICKUP -> PICKED_UP -> DELIVERED
    |           |           |
    (restaurant  (restaurant no-response  (no delivery partner found
    rejects ->   timeout -> auto-cancel   within window -> widen
    CANCELLED)   or reroute)               search radius, Ch 11's index)

```

```

Event-driven coordination (Order Service publishes, others react):
[Order Service] --event--> [Restaurant Service] (accept/prep)
                  --event--> [Matching Service, Ch 11] (assign courier)
                  --event--> [Notification Service, Ch 15] (keep all parties i
nformed)

```

Each downstream service reacts independently -- no synchronous blocking chain across all three parties.

4. Trade-offs Table

Decision	Alternative	Why this choice
Explicit order state machine	Free-form status string/flags	Prevents invalid/undefined order states; makes valid transitions explicit
Event-driven coordination across 3 parties	Order Service synchronously calls & waits on all three	One slow party can't block the others; matches Saga-style resilience
Timeout-and-fallback at each transition	Wait indefinitely for restaurant/courier response	Prevents orders getting permanently stuck when a party doesn't respond
Separate ETA-prediction service	Static/fixed delivery time estimate	Real-time inputs (traffic, prep time, courier distance) meaningfully improve UX

5. Common Interview Questions

Q1: "What happens if a restaurant never responds to an order?"

Model approach: Name an explicit timeout policy at the ACCEPTED transition — auto-cancel with refund, or automatic rerouting to a comparable restaurant — rather than leaving the order in an indefinite pending state.

Q2: "How is this different from designing the ride-hailing system in Chapter 11?"

Model approach: Name the added three-party coordination complexity directly — ride-hailing matches two parties (rider, driver); food delivery must additionally coordinate the restaurant's independent preparation timeline before the delivery-matching step from Chapter 11 even begins.

Q3: "How would you design ETA prediction without building a full ML system live in the interview?"

Model approach: Name the appropriate inputs (prep-time history, live courier distance/traffic, historical route timing) and scope it as a separate consuming service, showing awareness of the real

subsystem's shape without over-committing to modeling details out of scope for this interview.

6. Practice Exercises

- 1 Run the full six-step framework for "design a food delivery platform like Swiggy or Zomato," out loud, with a timer.
- 2 Draw the full order state machine including every failure/timeout branch, not just the happy path.
- 3 Design the widen-search-radius fallback for delivery partner matching: what specific parameters change, and how many times would you retry before escalating further?

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"System Design Interview: Design Zomato Swiggy Uber Eats DoorDash w/ a Senior Software Engineer"	Full mock interview directly covering this exact multi-party problem.
English	"System Design 6: Design Food Delivery Application like Zomato / Swiggy / Uber Eats HLD LLD"	Structured HLD/LLD breakdown of restaurant, order, and delivery coordination.
Hindi	"System Design of Zomato, Swiggy, DoorDash Food Delivery App Architecture Explained"	Hindi/Hinglish coverage of this domain's specific architecture.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Bilingual course covering multi-service coordination as part of the broader curriculum.

8. Key Takeaways

- Food delivery combines Chapter 11's geospatial matching with Chapter 10's order-correctness thinking, plus new three-party coordination.
- An explicit order state machine prevents invalid or undefined order states and makes valid transitions clear.
- Event-driven coordination across restaurant, delivery partner, and customer avoids one slow party blocking the others.
- Timeout-and-fallback policies at each state transition are a hard production requirement, since the platform doesn't directly control restaurants or couriers.
- ETA prediction is a real, separate subsystem consuming live and historical data, appropriately scoped rather than built live in an interview.
- Peaked lunch/dinner load patterns mean average-throughput estimates understate the real design target.
- This chapter is a genuine combination problem, rewarding candidates who can cite and connect earlier chapters (11, 10, 15) rather than solving it from scratch.